

React Hooks Cheat Sheet

`useState` and `useEffect` are hooks in React. Hooks are special functions in React that allow you to use built-in React features, like state, lifecycle, and context in functional components.

useState

```
const [state, setState] = useState(initialValue)
```

- **Purpose:** Add state to functional components. Used to build components that take inputs.
- **Parameters:**
 - `initialValue`: starting value (0 for counter, empty for input bars, etc...)
- **Returns:** Array with current state and a setter function (function providing a new state).
- **Example:**

```
const [count, setCount] = useState(0);
```

useEffect

```
useEffect(() => { ... }, [dependencies])
```

- **Purpose:** Run side effects when a specific input is given.
- **Parameters:**
 - **Dependencies:** the `useEffect` will run when `[dependencies]` change.
- **Example:**

```
useEffect(() => {  
  fetchData();  
}, []);
```

- **Explanation:**
 - this is used mostly for fetching data from APIs, databases, etc...
 - first runs after the component renders (is built in UI), and when a value in the dependency array(`[]`) changes, then `useEffect` re-runs and executes the function inside.
 - `fetchData()` is in `{}` of the `useEffect`, therefore when `useEffect` is called, the method `fetchData()` is executed.

useContext

```
const value = useContext(MyContext)
```

- **Description:** when you have nested components (components inside other components); when sending info from the most outside to inside component, without context, the info goes through each layer of components (which is a disaster). `context` is like a board which every component can access directly.

The outer component puts it on the board (`context`) and the inside component reads that board(`uses that context`).

- **Purpose:**
 - `MyContext`: `React.createContext()` creates context objects. `useContext` takes context object and returns its current value. (gives the information stored in the context to the component)
- **Example:**

```
// 1. Create the Context
const ThemeContext = React.createContext('light');

// 2. Provide the Context (in a parent component)
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

// 3. Consume the Context (in a child component)
function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div>Current theme: {theme}</div>; // "dark"
}
```

- **Explanation:**
 - `const ThemeContext = React.createContext('light')` creates a context named `light`, we want it as a backup plan if the `dark` context fails.
 - `ThemeContext.Provider value="dark"` is the one which provides the `dark` context.
 - `theme` will get the value of the `ThemeContext` to use it in the function.

Rules of Hooks

- **Only call hooks at the top level** (not inside loops, conditions, or nested functions)
- **Only call hooks from React functions** (not regular JavaScript functions)
- Use the `use` prefix for custom hooks.

Want to learn more? Read about `useState` and its application here: [Understanding useState in React](#)